

CSE 256 | Lecture 3: Linear Models - Loss Function

Ndapa Nakashole

January 13, 2026

Administrative Matters

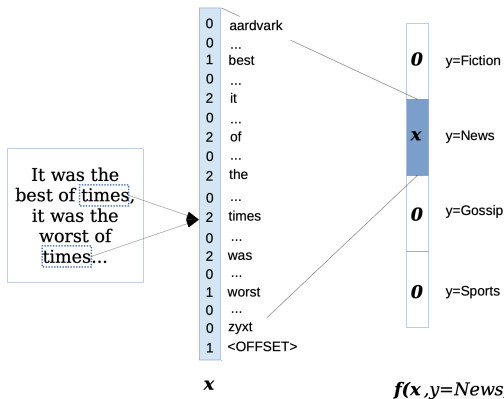
- ▶ **PA1** is out; due **January 30**
- ▶ Late submissions:
 - ◉ Up to 2 days late allowed
 - ◉ 15% penalty for 1 day late
 - ◉ 30% penalty for 2 days late

Course up to now ...

- ▶ Introduced one NLP task: Text Classification
- ▶ Talked about two simple models

Recap: Text Classification, Representation

- ▶ Categorize a piece of text into one of many predefined categories
- ▶ Representation: create a **numerical feature vector** from text, e.g., BoW

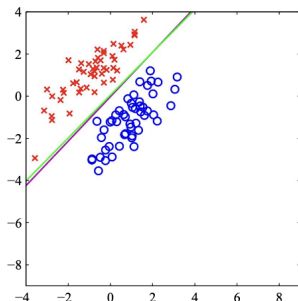


brief recap of Lecture 2 key ideas (due to lost audio):

Linear Classifier

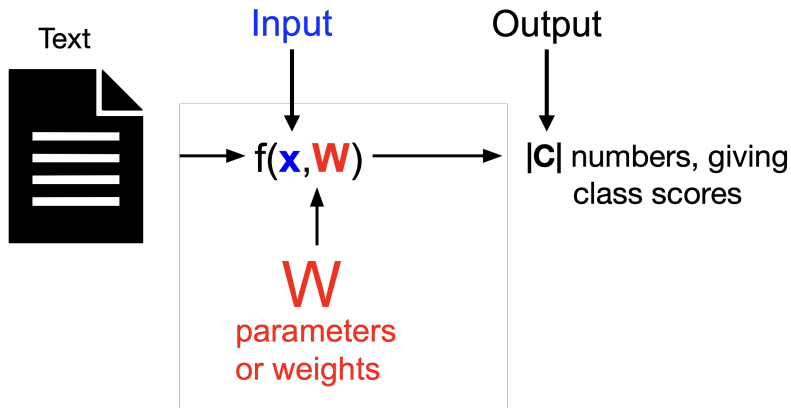
Linear Classifier

- ▶ **Predefined Model Complexity:** assume a specific form for the decision boundary
 - ⊙ straight line: defined by a small number of **parameters**



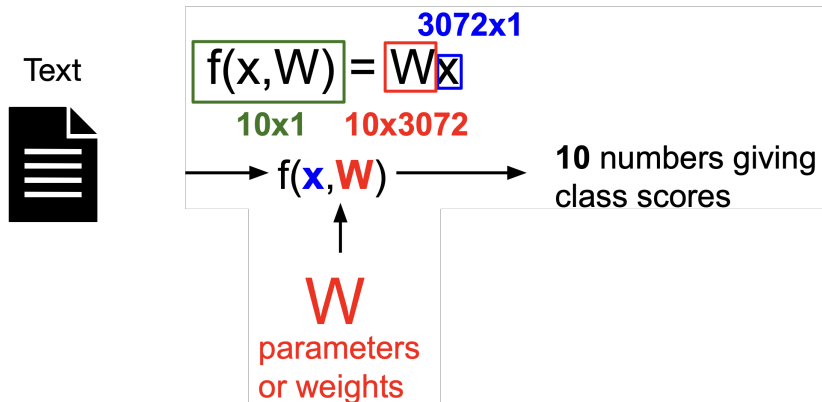
Linear Classifier: notation

A function f of the input features x , and the parameters W :



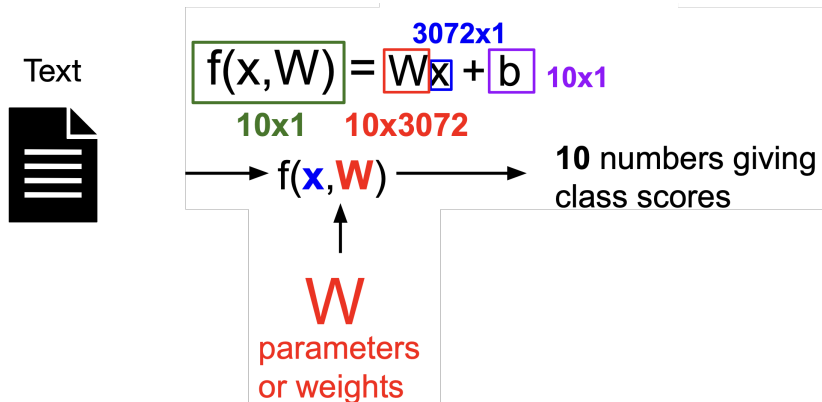
Linear Classifier: example

- ▶ Consider a 10 way classification problem
- ▶ 10 classes, 3072 features
- ▶ $f(x, W) = Wx$



Linear Classifier: bias term

- ▶ Consider a 10 way classification problem
- ▶ 10 classes, 3072 features (assume this is the vocabulary size)
- ▶ Bias term

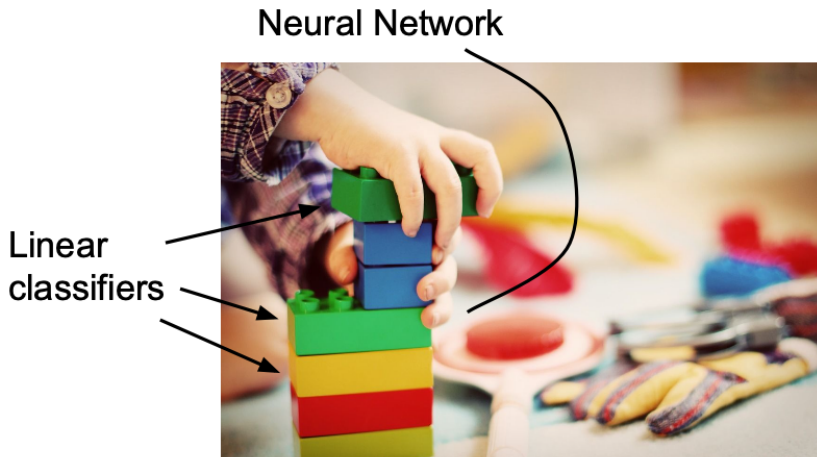


Bias term: intuition

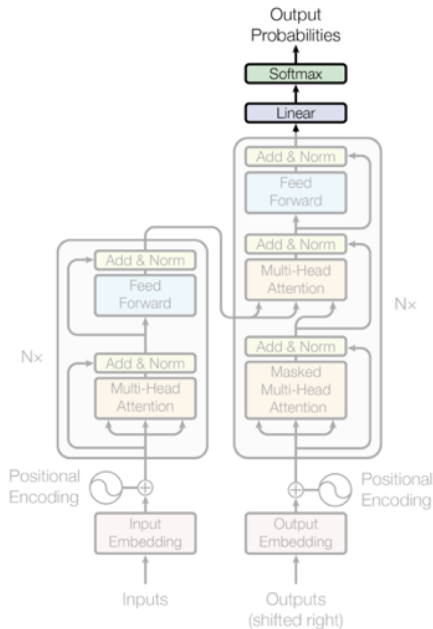
- ▶ Bias term helps tell the model: one class is much more common than others
 - ◉ e.g., consider spam vs non-spam email classification
 - ◉ 90% of emails are non-spam
 - ◉ The model should predict non-spam, unless the content of the email suggests otherwise

Linear Layers: a Building Block in Deep Learning

- ▶ Linear layers: a common building block in neural networks
- ▶ Neural Networks: consist of small functions, like lego blocks, that are easily stacked together to form a complex function



Linear Layers will keep coming back, e.g. in Transformers



Example: Email Classification

Example with a 4-dimensional email representation x , and 3 classes (Spam/Personal/Work)

0.2	-0.5	0.1	2.0
1.5	1.3	2.1	0.0
0	0.25	0.2	-0.3

W

3x4

56
231
24
2

x

4x1

+

1.1
3.2
-1.2

b

3x1

=

-96.8
437.9
61.95

Class scores

3x1

first class score: $(0.2 \cdot 56) + (-0.5 \cdot 231) + (0.1 \cdot 24) + (2.0 \cdot 2) = 11.2 - 115.5 + 2.4 + 4.0 = -97.9 + 1.1 = -96.8$

One interpretation: each row of W is a template for a class. The score is the similarity between the input and the template.

Where do the numbers come from?

Diagram illustrating the calculation of Class scores from a weight matrix W and input vector x .

Learned W (3x4):

0.2	-0.5	0.1	2.0
1.5	1.3	2.1	0.0
0	0.25	0.2	-0.3

Given x (4x1):

56
231
24
2

Learned b (3x1):

1.1
3.2
-1.2

Calculated Class scores (3x1):

-96.8
437.9
61.95

The calculation is shown as: $Wx + b = \text{Class scores}$

Which W is the best?

Today

- ① Softmax Classifier: from scores to probabilities
- ② Softmax Loss Function: measuring prediction quality
- ③ Why we need optimization (preview)

Probability: A Language for Uncertainty

Machine learning models are inherently uncertain.

- ▶ Real-world data is noisy, incomplete, and ambiguous
- ▶ Models rarely make perfectly certain predictions
- ▶ We need a principled way to express:
 - ⊙ not just *what* the model predicts
 - ⊙ but *how confident* it is in that prediction

Probability

Probability is the mathematical language we use to quantify uncertainty.
— *Larry Wasserman, All of Statistics*

Softmax Classifier

Linear classifier up to now:

$$\underbrace{s}_{\text{Unnormalized class scores}} = f(x_i; W) = \underbrace{Wx_i}_{\text{Weighted sum of inputs}}$$

These scores are not probabilities!

Softmax converts scores into probabilities:

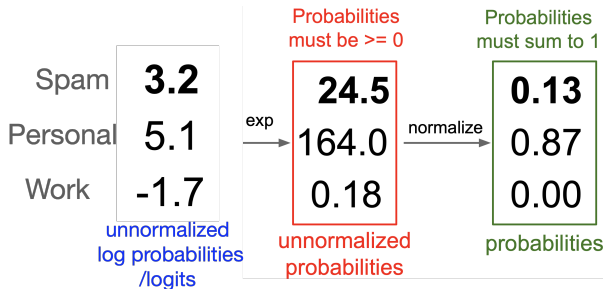
$$P(Y = k \mid X = x_i) = \frac{e^{s_k}}{\underbrace{\sum_j e^{s_j}}_{\text{Softmax}}}$$

Softmax: Numerical Example

$$s = f(x_i; W) = Wx_i$$

Want to interpret raw classifier scores as probabilities

$$P(Y = k \mid X = x_i) = \frac{e^{s_k}}{\underbrace{\sum_j e^{s_j}}_{\text{Softmax function}}}$$



Now we can talk about: **the likelihood of a document belonging to a particular class**

Loss Function

Loss: Conceptual Motivation

A loss function tells us how good or bad our current classifier is.

Intuition:

- ▶ High probability for the correct class $\rightarrow$ low loss
- ▶ Low probability for the correct class $\rightarrow$ high loss

Loss over a Dataset: Formal Definition

Given a dataset of examples:

$$\{(x_i, y_i)\}_{i=1}^N$$

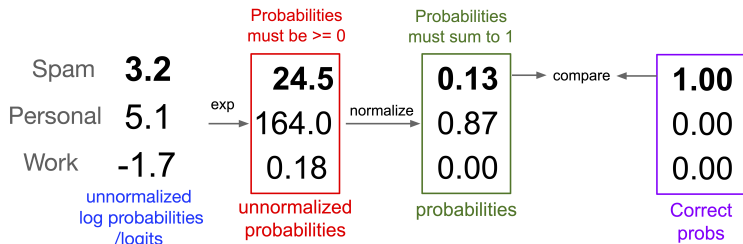
- ▶ x_i : text document
- ▶ y_i : true (gold) label

The total loss is the average loss over examples:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Softmax → Loss: Connecting the linear model to loss

- ▶ Key question:
 - ⊙ How much probability did the model assign to the correct class?
- ▶ Assume: true label is **SPAM**



Softmax → Loss: From probabilities, the loss follows naturally

- ▶ **Step 1: Fit the data** We want the model to assign high probability to the correct class

$$P(Y = y_i \mid X = x_i)$$

- ▶ **Step 2: Use log-probabilities** Probabilities can be very small; taking the log improves numerical stability

$$\log P(Y = y_i \mid X = x_i)$$

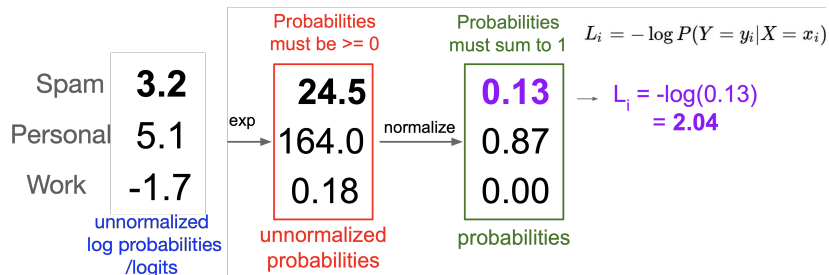
- ▶ **Step 3: Define a loss to minimize** We negate the log-probability so that minimizing loss $\Leftrightarrow$ maximizing likelihood

$$L_i = -\log P(Y = y_i \mid X = x_i)$$

This loss is called the **Softmax Loss**, also known as **Cross-Entropy Loss**.

Softmax → Loss: 1 training data point

Assume: true label is **SPAM**



Class Scores and Loss Calculation- Example

At initialization, values in W are small numbers

Class	score
Spam	0.05
Personal	0.01
Work	-0.03

Class	Step 1: Apply exp
Spam	$\exp(0.05) = 1.0513$
Personal	$\exp(0.01) = 1.0101$
Work	$\exp(-0.03) = 0.9704$

$$\sum = 1.0513 + 1.0101 + 0.9704 = 3.0318$$

Class	Step 2: Normalized Probabilities
Spam	$\frac{1.0513}{3.0318} = 0.35$
Personal	$\frac{1.0101}{3.0318} = 0.33$
Work	$\frac{0.9704}{3.0318} = 0.32$

Step 3: Loss for Spam:

$$L_i = -\log P(Y = \text{Spam} \mid X = x_i) = -\log(0.35) \approx 1.05$$

Maximum Likelihood Estimation

What we are doing is called
Maximum Likelihood Estimation (MLE).

Goal:

Choose parameters $\mathbf{W}$ that make the observed labels *as likely as possible* under the model.

$$\mathbf{W}_{ML} = \arg \max_{\mathbf{W}} \sum_{i=1}^n \log p(y_i \mid x_i; \mathbf{W})$$

Remember: want models to generalize

- ▶ **Generalization**: the ability of a model to perform well on unseen data
- ▶ But MLE can **Overfit**: fits the training data very well but fails to generalize
- ▶ **Regularization**: discourages the model from fitting the training data too closely
 - ◉ Especially important with many features or very flexible models

Pure MLE focuses on training performance; regularization helps with generalization.

In practice, we don't actually do pure maximum likelihood estimation (MLE).

Regularization

Regularization: adding a second term to the loss

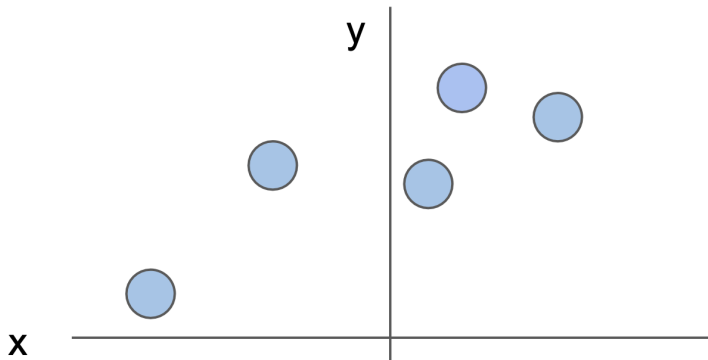
$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i; W), y_i) + \lambda R(W)$$

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

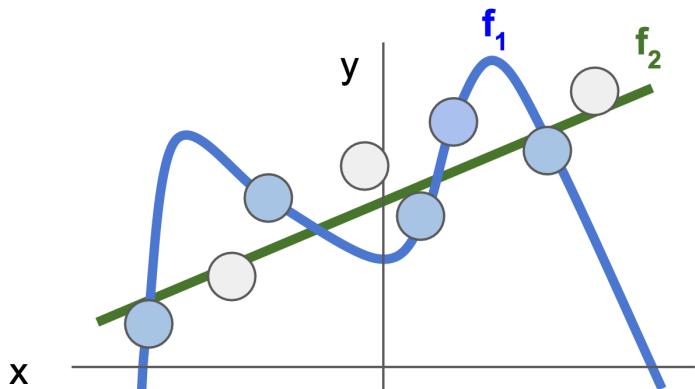
Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data

Regularization intuition: toy example training data



Regularization intuition: Prefer Simpler Models



- ▶ Regularization prefers **simpler, smoother models**
- ▶ Even if they fit the training data slightly worse

Softmax Loss with L2 Regularization

Recall: W contains all the learnable weights of the classifier

$$W = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1d} \\ w_{21} & w_{22} & \cdots & w_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{md} \end{bmatrix}$$

L2 Regularization: penalize large weights

$$R(W) = \|W\|_2^2 = \sum_{i=1}^m \sum_{j=1}^d w_{ij}^2$$

Regularization Strength

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data

λ = regularization strength (hyperparameter)

- ▶ $\lambda = 0$: No regularization
- ▶ $\lambda = \infty$: Only care about regularization
- ▶ $\lambda = 0.5$: Balance between fitting the data and regularization

Many ways to regularize models ...

Simple methods:

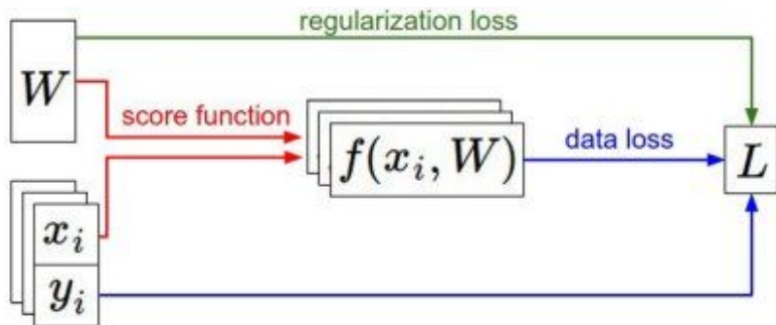
- ▶ L2: $R(W) = \sum_k \sum_l W_{k,l}^2$
- ▶ L1: $R(W) = \sum_k \sum_l |W_{k,l}|$

Complex methods:

- ▶ Dropout
- ▶ Batch normalization

Important idea is: regularization encodes a preference for simpler models.

Joint Loss Visualization



Basic (supervised) Machine Learning Recipe

Given: (many) input / output examples (x, y)

Find mapping f s.t. $\mathbf{y} \approx f(\mathbf{x})$

Define
$$L(\mathbf{W}) = \sum_{i=1}^N L(f_{\mathbf{w}}, \{\mathbf{x}_i, \mathbf{y}_i\}) + R(\mathbf{W})$$

learn by optimizing $\mathbf{W} = \arg \min_{\mathbf{W}} L(\mathbf{W})$

How does one minimize the loss to pick the best $\mathbf{W}$?

Questions ?